Full Stack Developer Technical Assessment

Project: Collaborative Kanban Board

Objective
Design and build a production-ready collaborative Kanban board application that demonstrates your ability to architect, implement, secure, document, test, and deploy a modern full-stack application. The assignment evaluates frontend development, backend APIs, database design, authentication, system design, deployment, code quality, and user experience.

Time Limit

2 Days (48 Hours)

Core Requirements

- User Authentication: Register, Login, Logout, JWT authentication, password hashing, protected routes, validation, secure error handling.
- Workspace Management: Create workspace, list workspaces, join via invite code, leave workspace.
- Kanban Board: Default columns (To Do, In Progress, Review, Done). CRUD tasks, drag/move between columns (API mandatory, UI drag-and-drop bonus), assignee, priority, due date, labels.
- Search & Filters: Search by title, filter by assignee/priority/status, sort by due date or creation date.
- Dashboard: Total tasks, tasks by status, priority distribution, overdue tasks, tasks assigned to current user.
- Responsive UI for desktop, tablet, and mobile.

Technical Requirements

- Frontend: React (preferred), Next.js, Vue, or Angular.
- Backend: Node.js + Express, NestJS, Spring Boot, Django, or equivalent.
- Database: PostgreSQL, MySQL, MongoDB, or another well-structured database.
- Architecture: Clean folder structure, reusable components, environment variables, error handling.
- Security: Input validation, JWT protection, hashed passwords, CORS, rate limiting (recommended).

Required REST APIs

- POST /auth/register
- POST /auth/login
- GET /workspaces
- POST /workspaces
- POST /workspaces/join
- GET /tasks

- POST /tasks
- PUT /tasks/:id
- DELETE /tasks/:id
- GET /dashboard

Deployment (Mandatory)

Deploy frontend, backend, and database. The application must be publicly accessible.

Submission Checklist

- GitHub repository with commit history
- Live frontend URL
- Live backend API URL
- README with setup instructions and architecture
- .env.example
- API Documentation (Swagger or Postman)
- Database schema/ER diagram
- Screenshots or demo video (optional)

Bonus Features

- Drag & Drop
- Role-Based Access Control
- Activity Log
- Comments
- Attachments
- Email Invitations
- Docker
- Unit/Integration Tests
- CI/CD
- Dark Mode
- WebSockets for live collaboration

Evaluation Rubric (100 Marks)

Functionality (30)
Backend/API Design (15)
Frontend/UI & UX (15)
Database Design (10)
Security & Validation (10)
Code Quality & Architecture (10)
Deployment & Documentation (5)
Testing & Bonus Features (5)

Assessment Expectations

Candidates should write clean, maintainable code, follow best practices, use meaningful commits, handle edge cases, and demonstrate production-level thinking. AI tools may be used for productivity, but candidates must understand and be able to explain every implementation decision during the interview.